

Implementação e Análise de Estruturas de Árvores Avançadas: PATRICIA, Trie, Treap e Splay

Felipe Silva
Engenharia de Computação
CEFET-MG Campus V
Divinópolis, Brasil
felipe.silva.03@aluno.cefetmg.br

Abstract—Este trabalho aborda o estudo de estruturas de dados hierárquicas não convencionais, explorando alternativas às tradicionais árvores de busca. O objetivo principal consiste em integrar fundamentação teórica, implementação computacional, análise de complexidade, visualização estrutural e avaliação experimental de cinco estruturas específicas: Árvore PATRICIA (Radix Tree compacta), Trie (Árvore de Prefixos), Árvore Splay, Árvore Treap (Tree + Heap) e KD-Tree. Por meio de uma abordagem comparativa, o estudo visa evidenciar os mecanismos de funcionamento, as vantagens e as limitações de cada estrutura, justificando tecnicamente sua aplicação em diferentes cenários computacionais.

Index Terms—PATRICIA, TRIE, TREAP, SPLAY, KD-Tree

I. INTRODUÇÃO E FUNDAMENTAÇÃO TEÓRICA

A organização estrutural e a recuperação eficiente de dados são desafios centrais na ciência da computação. Tradicionalmente, as Árvores Binárias de Busca (BST) e suas variantes estritamente balanceadas, como a Árvore AVL, são adotadas como o padrão para essas tarefas, garantindo complexidade de tempo logarítmica em cenários gerais. No entanto, à medida que a natureza dos dados e os padrões de acesso se diversificam, as abordagens clássicas revelam limitações de desempenho e flexibilidade.

Após a consolidação dos estudos em estruturas balanceadas clássicas, este trabalho avança para a exploração de árvores não convencionais. Tais estruturas diferem substancialmente entre si quanto aos conceitos de organização, armazenamento e balanceamento.

Para fins de análise comparativa, as estruturas foram divididas em três categorias metodológicas principais: (I) árvores focadas no processamento de *strings* e prefixos (PATRICIA e Trie); (II) árvores de busca binária baseadas em heurísticas e probabilidades (Splay e Treap); e (III) estruturas de particionamento espacial (KD-Tree).

O estudo detalha as operações fundamentais de cada estrutura, apresentando pseudocódigos e mecanismos de reorganização. As implementações completas foram desenvolvidas em C++ e disponibilizadas em repositório no GitHub [1]. Com isso, busca-se fundamentar tecnicamente as vantagens, limitações e critérios de escolha de cada árvore.

II. PROJETO E IMPLEMENTAÇÃO DAS ESTRUTURAS

A. PATRICIA Tree

1) *Contexto*: A Árvore PATRICIA (Radix Tree compacta) fundamenta-se na representação eficiente de chaves por meio da compressão de caminhos e compartilhamento de prefixos. Diferentemente de uma Trie padrão, cada nó armazena blocos completos de caracteres. O atributo lógico `endWord` indica o término de uma palavra válida. O mecanismo central baseia-se na divisão (*split*) de nós quando ocorrem divergências parciais de prefixos.

2) *Nó*: O registro estrutural que compõe a árvore armazena o bloco de caracteres da aresta, a flag de fim de palavra e o mapeamento de ponteiros.

Algorithm 1 Estrutura do Nó (RadixTreeNode)

```
1: Struct RadixTreeNode:  
2:   content  $\leftarrow$  String  
3:   endWord  $\leftarrow$  Boolean  
4:   children  $\leftarrow$  Map(Char, RadixTreeNode*)
```

3) *Busca*: A operação de busca percorre a árvore comparando os blocos de caracteres armazenados nos nós com os prefixos da string de entrada.

Algorithm 2 Busca - PATRICIA

```
1: procedure SEARCH(node, word)
2:   current  $\leftarrow$  node
3:   remaining  $\leftarrow$  word
4:   while remaining  $\neq \emptyset$  do
5:     firstChar  $\leftarrow$  remaining[0]
6:     if  $\neg$ current.children.count(firstChar) then
7:       return false
8:     end if
9:     child  $\leftarrow$  current.children[firstChar]
10:    if remaining.starts_with(child.content) then
11:      remaining  $\leftarrow$  remaining.substr(child.content.length())
12:      current  $\leftarrow$  child
13:    else
14:      return false
15:    end if
16:  end while
17:  return current.endWord
18: end procedure
```

4) *Inserção*: A inserção identifica o prefixo comum mais longo entre a palavra e o nó atual, fracionando-os em sufixos. O comportamento divide-se em quatro casos:

- **Caso 1:** Sufixos vazios; ativa *endWord*.
- **Caso 2:** Sufixo do nó vazio; delega para o filho ou cria folha.
- **Caso 3:** Nova palavra é prefixo; encurta o nó e rebaixa o restante.
- **Caso 4:** Divergência total; divide o nó e cria duas ramificações.

Algorithm 3 Inserção - PATRICIA

```
1: procedure INSERT(node, word)
2:   if SEARCH(node, word) then return
3:   end if
4:   count  $\leftarrow$  0
5:   while count < len(node.content) and count < len(word) do
6:     if node.content[count]  $\neq$  word[count] then
7:       break
8:     count  $\leftarrow$  count + 1
9:   end while
10:  prefix  $\leftarrow$  node.content[0...count - 1]
11:  suffixOld  $\leftarrow$  node.content[count...end]
12:  suffixNew  $\leftarrow$  word[count...end]
13:  if suffixOld =  $\emptyset$  and suffixNew =  $\emptyset$  then
14:    node.endWord  $\leftarrow$  true
15:    return
16:  end if
17:  if suffixOld =  $\emptyset$  and suffixNew  $\neq \emptyset$  then
18:    char  $\leftarrow$  suffixNew[0]
19:    if node.children.count(char) then
20:      INSERT(node.children[char], suffixNew)
21:    else
22:      node.children[char]  $\leftarrow$  new Node(suffixNew, true)
23:    end if
24:    return
25:  end if
26:  if suffixOld  $\neq \emptyset$  and suffixNew =  $\emptyset$  then
27:    oldChild  $\leftarrow$  new Node(suffixOld, node.endWord)
28:    oldChild.children  $\leftarrow$  node.children
29:    node.content  $\leftarrow$  prefix
30:    node.endWord  $\leftarrow$  true
31:    node.children.clear()
32:    node.children[suffixOld[0]]  $\leftarrow$  oldChild
33:    return
34:  end if
35:  newChild  $\leftarrow$  new Node(suffixNew, true)
36:  oldChild  $\leftarrow$  new Node(suffixOld, node.endWord)
37:  oldChild.children  $\leftarrow$  node.children
38:  node.content  $\leftarrow$  prefix
39:  node.endWord  $\leftarrow$  false
40:  node.children.clear()
41:  node.children[suffixNew[0]]  $\leftarrow$  newChild
42:  node.children[suffixOld[0]]  $\leftarrow$  oldChild
43: end procedure
```

5) *Remoção*: A remoção localiza a chave e aplica três cenários:

- **Caso 1:** Sem filhos; deleta o nó e retorna null.
- **Caso 2:** Único filho; funde o conteúdo e libera o redundante.
- **Caso 3:** Múltiplos filhos; desativa a flag *endWord*.

Algorithm 4 Remoção - PATRICIA

```
1: procedure REMOVE(node, word)
2:   if  $\neg$ SEARCH(node, word) then
3:     return null
4:   end if
5:   if word = node.content then
6:     if node.children.empty() then
7:       delete node
8:       return null
9:     end if
10:    if node.children.size() = 1 then
11:      onlyChild  $\leftarrow$  unico filho
12:      node.content  $\leftarrow$  node.content +
onlyChild.content
13:      node.children  $\leftarrow$  onlyChild.children
14:      node.endWord  $\leftarrow$  onlyChild.endWord
15:      delete onlyChild
16:      return node
17:    end if
18:    node.endWord  $\leftarrow$  false
19:    return node
20:  end if
21:  count  $\leftarrow$  0
22:  while count < len(node.content) and count <
len(word) do
23:    if node.content[count]  $\neq$  word[count] then
24:      break
25:    end if
26:    count  $\leftarrow$  count + 1
27:  end while
28:  suffixNew  $\leftarrow$  word[count...end]
29:  char  $\leftarrow$  suffixNew[0]
30:  node.children[char]  $\leftarrow$ 
REMOVE(node.children[char], suffixNew)
31:  if node.children[char] = null then
32:    node.children.erase(char)
33:  end if
34:  return node
35: end procedure
```

6) *Impressão*: O percurso utiliza Busca em Profundidade (DFS) para concatenar os blocos dos nós acumulativamente.

Algorithm 5 Percurso - PATRICIA

```
1: procedure PRINT(node, prefix, list)
2:   acc  $\leftarrow$  prefix + node.content
3:   if node.endWord then
4:     list.push_back(acc)
5:   end if
6:   for each (char, child) in node.children do
7:     PRINT(child, acc, list)
8:   end for
9: end procedure
```

B. Trie (Árvore de Prefixos)

1) *Contexto*: A Árvore Trie organiza chaves de caracteres de forma hierárquica, decompondo cada palavra em caracteres individuais onde cada nó representa um caractere e seus filhos direcionam para os caracteres subsequentes, permitindo o compartilhamento de prefixos comuns.

2) *Nó*: Cada nó armazena uma flag de término de palavra e um mapa de ponteiros direcionado aos caracteres filhos.

Algorithm 6 Estrutura do Nó (TrieTreeNode)

```
1: Struct TrieTreeNode:
2:   endWord  $\leftarrow$  Boolean
3:   children  $\leftarrow$  Map(Char, TrieTreeNode*)
4: procedure CONSTRUTOR(endWord = false)
5:   this.endWord  $\leftarrow$  endWord
6:   this.children  $\leftarrow$   $\emptyset$ 
7: end procedure
```

3) *Busca*: A busca consome a palavra caractere por caractere de forma recursiva até exauri-la, retornando o valor booleano da flag endWord do nó final ou falso caso algum caractere não seja encontrado.

Algorithm 7 Busca - Trie

```
1: procedure SEARCH(node, word)
2:   if word.empty() then
3:     return node.endWord
4:   end if
5:   firstChar  $\leftarrow$  word[0]
6:   if  $\neg$ node.children.count(firstChar) then
7:     return false
8:   end if
9:   child  $\leftarrow$  node.children[firstChar]
10:  return SEARCH(child, word.substr(1))
11: end procedure
```

4) *Inserção*: A inserção consome a palavra caractere por caractere descendo recursivamente pela árvore. O comportamento divide-se em dois casos:

- **Caso 1 (Fim da Palavra)**: String vazia; ativa a flag endWord do nó atual.
- **Caso 2 (Criação de Ramo)**: Caractere ausente; instancia nova ramificação e prossegue inserindo os caracteres restantes.

Algorithm 8 Inserção - Trie

```
1: procedure INSERT(node, word)
2:   if word.empty() then
3:     node.endWord  $\leftarrow$  true
4:     return
5:   end if
6:   firstChar  $\leftarrow$  word[0]
7:   if  $\neg$ node.children.count(firstChar) then
8:     node.children[firstChar]  $\leftarrow$ 
       new TrieTreeNode(false)
9:   end if
10:  INSERT(node.children[firstChar], word.substr(1))
11: end procedure
```

5) *Remoção*: A remoção navega até o fim da palavra desativando a flag *endWord* e limpa nós redundantes (órfãos sem filhos) recursivamente de baixo para cima.

Algorithm 9 Remoção - Trie

```
1: procedure REMOVE(node, word)
2:   if word.empty() then
3:     node.endWord  $\leftarrow$  false
4:   else
5:     char  $\leftarrow$  word[0]
6:     if node.children.count(char) then
7:       node.children[char]  $\leftarrow$ 
        REMOVE(node.children[char], word.substr(1))
8:       if node.children[char] = null then
9:         node.children.erase(char)
10:      end if
11:    end if
12:  end if
13:  if  $\neg$ node.endWord and node.children.empty() then
14:    delete node
15:    return null
16:  end if
17:  return node
18: end procedure
```

6) *Impressão*: O percurso utiliza Busca em Profundidade (DFS) acumulando os caracteres pelos ramos até registrar as palavras terminadas.

Algorithm 10 Percurso - Trie

```
1: procedure PRINT(printerList, node, word)
2:   if node.endWord then
3:     printerList.push_back(word)
4:   end if
5:   for each (char, child) in node.children do
6:     acc  $\leftarrow$  word + char
7:     PRINT(printerList, child, acc)
8:   end for
9: end procedure
```

C. Árvore Treap (Tree + Heap)

1) *Contexto*: A Árvore Treap combina propriedades de Árvore Binária de Busca (*Tree*) e de Heap (*Heap*). Enquanto as chaves organizam-se segundo as regras da BST, cada nó recebe um peso gerado aleatoriamente que obedece à propriedade de Heap. Isso evita a degradação de desempenho em dados ordenados, mantendo o balanceamento esperado em $\mathcal{O}(\log n)$.

2) *Nó*: Cada nó armazena uma chave de conteúdo, uma prioridade aleatória e ponteiros para os filhos esquerdo e direito.

Algorithm 11 Estrutura do Nó (TreapNode)

```
1: Struct TreapNode:
2:   content  $\leftarrow$  Integer
3:   weight  $\leftarrow$  Integer  $\triangleright$  Prioridade aleatória
4:   esq  $\leftarrow$  TreapNode*
5:   dir  $\leftarrow$  TreapNode*
```

3) *Busca*: A busca navega recursivamente comparando a chave desejada com o conteúdo do nó atual, direcionando-se à subárvore correspondente segundo as regras de uma BST.

Algorithm 12 Busca - Treap

```
1: procedure SEARCH(root, valueNode)
2:   if root = null or root.content = valueNode then
3:     return root
4:   end if
5:   if root.content > valueNode then
6:     return SEARCH(root.esq, valueNode)
7:   else
8:     return SEARCH(root.dir, valueNode)
9:   end if
10: end procedure
```

4) *Inserção*: A inserção ocorre como em uma BST comum, seguida de verificação de violação da propriedade de Heap. O comportamento divide-se em dois casos:

- **Caso 1 (Violação Esquerda)**: O peso do filho esquerdo é maior que o do pai; executa-se rotação à direita.
- **Caso 2 (Violação Direita)**: O peso do filho direito é maior que o do pai; executa-se rotação à esquerda.

Algorithm 13 Inserção - Treap

```
1: procedure INSERT(root, node)
2:   if root = null then
3:     return node
4:   end if
5:   if root.content > node.content then
6:     root.esq ← INSERT(root.esq, node)
7:     if root.esq ≠ null and root.weight <
       root.esq.weight then
8:       root ← rotacaoDir(root)
9:     end if
10:    root.dir ← INSERT(root.dir, node)
11:    if root.dir ≠ null and root.weight <
       root.dir.weight then
12:      root ← rotacaoEsq(root)
13:    end if
14:  end if
15:  return root
16: end procedure
```

Algorithm 14 Remoção - Treap

```
1: procedure REMOVE(root, valueNode)
2:   if root = null then
3:     return null
4:   end if
5:   if valueNode < root.content then
6:     root.esq ← REMOVE(root.esq, valueNode)
7:     root.dir ← REMOVE(root.dir, valueNode)
8:   else
9:     if root.esq = null and root.dir = null then
10:      delete root
11:      return null
12:      aux ← root.dir
13:      delete root
14:      return aux
15:      root.dir = null
16:      aux ← root.esq
17:      delete root
18:      return aux
19:    else
20:      if root.esq.weight > root.dir.weight then
21:        root ← rotacaoDir(root)
22:        root.dir ← REMOVE(root.dir, valueNode)
23:      else
24:        root ← rotacaoEsq(root)
25:        root.esq ← REMOVE(root.esq, valueNode)
26:      end if
27:    end if
28:  return root
29: end procedure
```

6) *Impressão*: O percurso simétrico (*In-Order*) extrai as chaves ordenadamente.

5) *Remoção*: A remoção localiza o nó e reorganiza a estrutura com base em seus filhos. Divide-se em três cenários:

- **Caso 1 (Folha)**: Sem filhos; deleta o nó diretamente e retorna null.
- **Caso 2 (Filho Único)**: Possui apenas um filho; remove o nó e conecta o filho ao pai.
- **Caso 3 (Dois Filhos)**: Compara os pesos dos filhos; o de maior peso é rotacionado para cima, convertendo o nó alvo em folha ou nó simples.

Algorithm 15 Percurso In-Order - Treap

```
1: procedure PRINTINORDER(root, printer)
2:   if root = null then
3:     return
4:   end if
5:   PRINTINORDER(root.esq, printer)
6:   printer.push_back(root.content)
7:   PRINTINORDER(root.dir, printer)
8: end procedure
```

D. Árvore Splay

1) *Contexto*: A Árvore Splay é uma árvore binária de busca autoajustável que soluciona o problema de custos elevados em consultas repetitivas. Seu diferencial reside na heurística *Splay*: toda vez que um nó é acessado, ele é movido iterativamente para a raiz da árvore por meio de rotações estruturadas. Desse modo, consultas frequentes a um mesmo elemento tornam-se

extremamente rápidas ($\mathcal{O}(1)$ na raiz), aproveitando o princípio da localidade temporal dos dados.

2) *Nó*: Cada nó da Splay armazena o conteúdo da chave, ponteiros para os filhos esquerdo e direito, e um ponteiro explícito para o nó pai (*parent*), essencial para rastrear os ancestrais durante as rotações ascendentes.

Algorithm 16 Estrutura do Nó (SplayNode)

```

1: Struct SplayNode:
2:   content  $\leftarrow$  Integer
3:   esq  $\leftarrow$  SplayNode*
4:   dir  $\leftarrow$  SplayNode*
5:   parent  $\leftarrow$  SplayNode*  $\triangleright$  Ponteiro essencial para o
   pai

```

3) *A Operação Splay (Heurística de Acesso)*: A função *splay* reorganiza a árvore elevando um nó até a raiz através de três casos fundamentais de rotação baseados na configuração relativa entre o nó alvo, seu pai e seu avô:

- **Caso 1 (Zig)**: O pai do nó é a própria raiz da árvore (sem avô). Aplica-se uma rotação simples na direção oposta ao nó.
- **Caso 2 (Zig-Zig)**: O nó e seu pai estão na mesma direção linear em relação ao avô (ambos filhos à esquerda ou à direita). Executa-se uma rotação no avô seguida de uma rotação no pai.
- **Caso 3 (Zig-Zag)**: O nó e seu pai estão em direções opostas em relação ao avô (formando um "cotovelo"). Executam-se duas rotações centradas no próprio nó (uma no nível do pai e outra no avô).

Algorithm 17 Operação Splay - Splay Tree

```

1: procedure SPLAY(node)
2:   while node.parent  $\neq$  null do
3:     pai  $\leftarrow$  node.parent
4:     avo  $\leftarrow$  pai.parent
5:     if avo = null then  $\triangleright$  Caso Zig
6:       if node = pai.esq then
7:         rotacaoDir(pai)
8:       else
9:         rotacaoEsq(pai)
10:      end if pai.esq = node and avo.esq = pai  $\triangleright$ 
      Caso Zig-Zig Esquerdo
11:      rotacaoDir(avo)
12:      rotacaoDir(pai) pai.dir = node and
      avo.dir = pai  $\triangleright$  Caso Zig-Zig Direito
13:      rotacaoEsq(avo)
14:      rotacaoEsq(pai) pai.dir = node and
      avo.esq = pai  $\triangleright$  Caso Zig-Zag Esquerdo-Direito
15:      rotacaoEsq(pai)
16:      rotacaoDir(avo)
17:      else  $\triangleright$  Caso Zig-Zag Direito-Esquerdo
18:        rotacaoDir(pai)
19:        rotacaoEsq(avo)
20:      end if
21:    end while
22: end procedure

```

4) *Busca*: A busca realiza a navegação binária padrão. Caso o elemento seja encontrado, a heurística *splay* é acionada para elevá-lo à raiz. Se o elemento não existir, o último nó visitado antes da falha é promovido à raiz.

Algorithm 18 Busca - Splay

```

1: procedure SEARCH(node, anterior, valor)
2:   if node = null then
3:     if anterior  $\neq$  null then
4:       SPLAY(anterior)
5:     end if
6:     return null
7:   end if
8:   if valor = node.content then
9:     SPLAY(node)
10:    return node
11:   end if
12:   if valor > node.content then
13:     return SEARCH(node.dir, node, valor)
14:   else
15:     return SEARCH(node.esq, node, valor)
16:   end if
17: end procedure

```

5) *Inserção*: A inserção posiciona o novo nó conforme as regras da BST e aplica a operação *splay* nele para torná-lo a nova raiz da árvore.

Algorithm 19 Inserção - Splay

```
1: procedure INSERT(node, anterior, valor)
2:   if node = null then
3:     newNode  $\leftarrow$  new SplayNode(valor, null, null, anterior)
4:     if valor < anterior.content then
5:       anterior.esq  $\leftarrow$  newNode
6:     else
7:       anterior.dir  $\leftarrow$  newNode
8:     end if
9:     SPLAY(newNode)
10:    return
11:  end if
12:  if valor > node.content then
13:    INSERT(node.dir, node, valor)
14:  else
15:    INSERT(node.esq, node, valor)
16:  end if
17: end procedure
```

6) *Remoção*: A remoção busca pelo valor (promovendo-o à raiz via *splay*), isolando-o e dividindo a árvore em duas subárvores (*L* e *R*). O maior elemento de *L* é trazido à sua raiz por *splay*, e *R* é anexado a ele.

Algorithm 20 Remoção - Splay

```
1: procedure REMOVE(valor)
2:   noRemover  $\leftarrow$  SEARCH(valor)
3:   if noRemover = null or noRemover.content  $\neq$ 
   valor then
4:     return
5:   end if
6:   L  $\leftarrow$  noRemover.esq
7:   R  $\leftarrow$  noRemover.dir
8:   if L  $\neq$  null then L.parent  $\leftarrow$  null
9:   end if
10:  if R  $\neq$  null then R.parent  $\leftarrow$  null
11:  end if
12:  delete noRemover
13:  if L = null and R = null then
14:    root  $\leftarrow$  null
15:    return
16:  end if
17:  if L = null then
18:    root  $\leftarrow$  R
19:    return
20:  end if
21:  if R = null then
22:    root  $\leftarrow$  L
23:    return
24:  end if
25:  aux  $\leftarrow$  L
26:  while aux.dir  $\neq$  null do
27:    aux  $\leftarrow$  aux.dir
28:  end while
29:  SPLAY(aux)
30:  aux.dir  $\leftarrow$  R
31:  R.parent  $\leftarrow$  aux
32:  root  $\leftarrow$  aux
33: end procedure
```

7) *Impressão*: O percurso simétrico (*In-Order*) extrai as chaves em ordem crescente.

Algorithm 21 Percurso In-Order - Splay

```
1: procedure PRINTINORDER(node)
2:   if node  $\neq$  null then
3:     PRINTINORDER(node.esq)
4:     print(node.content)
5:     PRINTINORDER(node.dir)
6:   end if
7: end procedure
```

E. KD-Tree (*k*-dimensional Tree)

1) *Contexto*: A KD-Tree (Árvore *k*-dimensional) é uma estrutura de dados voltada ao particionamento espacial de pontos em um espaço multidimensional (*k* dimensões). Diferentemente das árvores binárias de busca convencionais que comparam chaves em uma única direção escalar, a KD-Tree alterna os eixos de comparação (como *X* e *Y* em duas

dimensões) a cada nível da árvore. O plano de corte divide o espaço em regiões de semi-espacos, permitindo consultas geométricas eficientes, tais como busca por proximidade, abrangência de regiões e k-vizinhos mais próximos (k -NN), mitigando o problema da maldimensionalidade em buscas espaciais.

2) *Nó*: Cada nó da KD-Tree armazena um vetor dinâmico de inteiros representando as coordenadas espaciais do ponto (*point*), além de dois ponteiros direcionados aos filhos esquerdo e direito para as ramificações espaciais.

Algorithm 22 Estrutura do Nó (KNode)

```

1: Struct KNode:
2:   point  $\leftarrow$  Vector(Integer)  $\triangleright$  Coordenadas espaciais do
   ponto
3:   esq  $\leftarrow$  KNode*
4:   dir  $\leftarrow$  KNode*

```

3) *Busca*: A busca de um ponto na KD-Tree verifica se as coordenadas coincidem. Caso contrário, calcula-se o eixo atual com base na profundidade ($\text{depth} \pmod k$) e o algoritmo desce pela subárvore esquerda (se a coordenada testada for menor) ou direita.

Algorithm 23 Busca - KD-Tree

```

1: procedure SEARCH(node, point, depth)
2:   if node = null then
3:     return false
4:   end if
5:   if AREPOINTS EQUAL(node.point, point) then
6:     return true
7:   end if
8:   cd  $\leftarrow$  depth  $\pmod k$ 
9:   if point[cd] < node.point[cd] then
10:    return SEARCH(node.esq, point, depth + 1)
11:  else
12:    return SEARCH(node.dir, point, depth + 1)
13:  end if
14: end procedure

```

4) *Inserção*: A inserção desce recursivamente alternando os eixos de comparação por nível. O comportamento divide-se em dois casos:

- **Caso 1 (Subárvore Vazia)**: Encontra-se a posição correta de inserção; um novo nó contendo o vetor de coordenadas é alocado e retornado.
- **Caso 2 (Descida Espacial)**: Compara-se a coordenada do eixo atual (*cd*) e direciona-se a inserção para a subárvore esquerda (se menor) ou direita (se maior ou igual).

Algorithm 24 Inserção - KD-Tree

```

1: procedure INSERT(node, point, depth)
2:   if node = null then
3:     return new KNode(point)
4:   end if
5:   cd  $\leftarrow$  depth  $\pmod k$ 
6:   if point[cd] < node.point[cd] then
7:     node.esq  $\leftarrow$  INSERT(node.esq, point, depth + 1)
8:   else
9:     node.dir  $\leftarrow$  INSERT(node.dir, point, depth + 1)
10:  end if
11:  return node
12: end procedure

```

5) *Remoção*: A remoção localiza o ponto desejado alternando eixos por profundidade. Ao encontrá-lo, a reestruturação lida com três cenários:

- **Caso 1 (Filho à Direita Disponível)**: Busca-se o nó com o menor valor na dimensão atual dentro da subárvore direita, copia-se seu conteúdo para o nó atual e remove-se o sucessor recursivamente.
- **Caso 2 (Apenas Filho à Esquerda)**: Como não há subárvore direita, busca-se o menor elemento na subárvore esquerda, copia-se para o nó atual, reloca-se a subárvore esquerda para o ponteiro direito (mantendo as propriedades da árvore) e limpa-se o ponteiro esquerdo.
- **Caso 3 (Nó Folha)**: O nó não possui filhos; é diretamente deletado e retorna-se **null**.

Algorithm 25 Remoção - KD-Tree

```
1: procedure REMOVE(node, point, depth)
2:   if node = null then
3:     return null
4:   end if
5:    $cd \leftarrow \text{depth} \pmod k$ 
6:   if AREPOINTSEQUAL(node.point, point) then
7:     if node.dir  $\neq$  null then
8:        $\text{min} \leftarrow \text{FINDMIN}(\text{node.dir}, cd, \text{depth} + 1)$ 
9:       node.point  $\leftarrow$  min.point
10:      node.dir  $\leftarrow$  REMOVE(node.dir, min.point, depth + 1)
11:      node.esq  $\neq$  null
12:       $\text{min} \leftarrow \text{FINDMIN}(\text{node.esq}, cd, \text{depth} + 1)$ 
13:      node.point  $\leftarrow$  min.point
14:      node.dir  $\leftarrow$  REMOVE(node.esq, min.point, depth + 1)
15:      node.esq  $\leftarrow$  null
16:     else
17:       delete node
18:       return null
19:     end if
20:     return node
21:   end if
22:   if point[cd] < node.point[cd] then
23:     node.esq  $\leftarrow$  REMOVE(node.esq, point, depth + 1)
24:   else
25:     node.dir  $\leftarrow$  REMOVE(node.dir, point, depth + 1)
26:   end if
27:   return node
28: end procedure
```

6) *Impressão*: O percurso simétrico (*In-Order*) explora os ramos e imprime as coordenadas de cada ponto estruturadamente.

Algorithm 26 Percurso In-Order - KD-Tree

```
1: procedure PRINTINORDER(node)
2:   if node  $\neq$  null then
3:     PRINTINORDER(node.esq)
4:     print(node.point)
5:     PRINTINORDER(node.dir)
6:   end if
7: end procedure
```

III. ANÁLISE DE COMPLEXIDADE E COMPARAÇÃO TEÓRICA

A avaliação assintótica das estruturas abordadas neste estudo revela abordagens fundamentalmente distintas para a organização e recuperação de dados, distanciando-os dos padrões tradicionais das árvores binárias de busca convencionais e das árvores AVL.

A. *Árvore PATRICIA (Radix Tree)*

1) *Complexidade de Tempo*: Para as operações de busca, inserção e remoção, o custo computacional é de $\mathcal{O}(m)$ no

melhor, médio e pior caso, onde m representa o comprimento (número de caracteres) da chave processada. O percurso navega de forma estruturada pelos blocos de caracteres armazenados nos nós e nas arestas, consumindo os caracteres correspondentes a cada nível de forma independente do volume total de chaves (n).

2) *Complexidade de Espaço*: O consumo espacial é de $\mathcal{O}(M)$, onde M representa o somatório cumulativo do comprimento de todas as chaves inseridas, otimizado pela compressão de caminhos e compartilhamento de prefixos.

B. *Árvore Trie*

1) *Complexidade de Tempo*: As ações de remoção, busca e inserção possuem complexidade $\mathcal{O}(m)$, sendo m o comprimento da palavra, pois o algoritmo inspeciona um caractere por nível de maneira estritamente sequencial.

2) *Complexidade de Espaço*: A complexidade espacial é de $\mathcal{O}(N \times m)$ no pior caso. A alocação de nós separados para cada caractere isolado, e não de forma compactada, gera um gasto excessivo e pesado de ponteiros.

C. *Árvore Treap*

1) *Complexidade de Tempo*: As operações de busca, inserção e remoção apresentam pior caso de $\mathcal{O}(n)$ e caso médio/melhor de $\mathcal{O}(\log n)$. O uso de pesos gerados estocasticamente garante estatisticamente uma altura logarítmica e um balanceamento probabilístico efetivo, com probabilidade de degradação tendendo a zero.

2) *Complexidade de Espaço*: O custo espacial é de $\mathcal{O}(n)$, exigindo a alocação de exatamente um nó por elemento de forma análoga às árvores BST e AVL convencionais.

D. *Árvore Splay*

1) *Complexidade de Tempo*: As operações de busca, inserção e remoção possuem pior caso em $\mathcal{O}(n)$ para uma única operação isolada, mas alcançam $\mathcal{O}(\log n)$ amortizado em uma sequência de operações. A heurística *splay* eleva os nós frequentemente consultados à raiz, garantindo acesso em $\mathcal{O}(1)$ para consultas repetitivas.

2) *Complexidade de Espaço*: A complexidade espacial é de $\mathcal{O}(n)$, pois aloca um nó para cada elemento armazenado sem metadados excessivos de balanceamento estático.

E. *KD-Tree (k-dimensional Tree)*

1) *Complexidade de Tempo*: A construção inicial (*build*) apresenta custo de $\mathcal{O}(n \log n)$ quando balanceada. Para busca, inserção e remoção, o caso médio atinge $\mathcal{O}(\log n)$ ao alternar os eixos de corte por profundidade ($\text{depth} \pmod k$), embora possa degenerar para $\mathcal{O}(n)$ se a inserção for desbalanceada.

2) *Complexidade de Espaço*: O custo espacial é estritamente $\mathcal{O}(n)$, dimensionando-se linearmente com o número de pontos multidimensionais armazenados.

TABLE I
COMPARATIVO DE DESEMPENHO PRÁTICO: PATRICIA VS. TRIE

Estrutura	Inserção Alfabética (10^6)	Inserção Aleatória (10^6)	Busca (10^3)	Remoção (10^3)	Tempo Total
PATRICIA	445 ms	2658 ms	1 ms	1 ms	3105 ms
Trie	910 ms	1765 ms	1 ms	1 ms	2677 ms

TABLE II
COMPARATIVO DE DESEMPENHO PRÁTICO: TREAP VS. SPLAY

Estrutura	Ins. Crescente (10^6)	Ins. Decrescente (10^6)	Ins. Aleatória (10^6)	Busca (10^3)	Remoção (10^3)	Busca Repetida (10^5)	Tempo Total
Treap	3156 ms	3104 ms	5210 ms	144 ms	1 ms	1 ms	11616 ms
Splay	43 ms	37 ms	2599 ms	204 ms	1 ms	0 ms	2884 ms

IV. METODOLOGIA EXPERIMENTAL E RESULTADOS

TABLE III
DESEMPENHO PRÁTICO DA KD-TREE

Operação	Inserção	Busca	Remoção (10^3)
KD-Tree	2130 ms	2 ms	3 ms

V. ANÁLISE CRÍTICA E DISCUSSÃO

A. Comparativo entre PATRICIA e Trie

Conforme observado nas definições teóricas e análises de complexidade, ao compararmos a Árvore PATRICIA e a Trie, os resultados experimentais evidenciam diferenças marcantes de desempenho conforme o padrão de entrada:

1) Operação de Inserção:

- **Inserção Alfabética (10^6):** A PATRICIA foi $2.04\times$ mais rápida (redução de 51.1% no tempo de execução, registrando 445 ms contra 910 ms da Trie). Esse ganho decorre da compactação de prefixos da PATRICIA, que agrupa blocos contínuos de caracteres em arestas únicas, mitigando a alocação excessiva de nós intermediários exigida pela Trie.
- **Inserção Aleatória (10^6):** O comportamento inverte-se, com a Trie registrando uma execução $1.50\times$ mais rápida do que a PATRICIA (melhoria de 33.6% , com 1765 ms frente a 2658 ms). A inserção aleatória fragmenta os prefixos, obrigando a PATRICIA a realizar constantes e custosas operações de divisão de nós (*splits*) e reconfigurações estruturais.

2) **Operações de Busca e Remoção:** Nas operações de busca (1 ms) e remoção (1 ms), os tempos computacionais foram altamente eficientes e estatisticamente equivalentes, apresentando paridade absoluta entre as estruturas ($1.0\times$ ou 0% de diferença estrutural no acesso final).

B. Comparativo entre Treap e Splay

Na avaliação entre a Árvore Treap e a Splay, os testes práticos demonstram o impacto direto das heurísticas de balanceamento:

1) Operação de Inserção:

- **Inserção Crescente e Decrescente (10^6):** A Splay demonstrou uma superioridade colossal, sendo $73.39\times$ mais rápida na inserção crescente (redução de 98.6% , com 43 ms contra 3156 ms) e $83.89\times$ superior na decrescente (redução de 98.8% , com 37 ms frente a 3104 ms).
- **Inserção Aleatória (10^6):** A Splay manteve-se $2.00\times$ mais rápida (melhoria de 50.1% , registrando 2599 ms contra 5210 ms).
- **Justificativa:** O custo de reestruturação da Splay restringe-se a rotações imediatas do novo nó na extremidade, enquanto a Treap carrega o pesado ônus estocástico de gerar prioridades e realizar varreduras e comparações contínuas de pesos para manter a propriedade de Heap.

2) Operações de Busca e Remoção:

- **Busca Unitária (10^3):** A Treap foi $1.41\times$ mais rápida que a Splay (vantagem de 29.4% , registrando 144 ms contra 204 ms). A Splay sofre um gargalo pontual devido à obrigação de executar a operação de *splay* (subir o nó até a raiz) a cada consulta avulsa.
- **Busca Repetitiva (10^5):** Ao testarmos consultas sucessivas aos mesmos dados, a Splay suplantou a Treap registrando 0 ms contra 1 ms, provando empírica e teoricamente sua capacidade de amortizar o custo temporal e converter consultas frequentes para $O(1)$.
- **Remoção (10^3):** Ambas empataram tecnicamente com 1 ms ($1.0\times$).

C. Desempenho e Comportamento da KD-Tree

Os resultados experimentais da KD-Tree apontam que a etapa de inserção espacial consome a totalidade expressiva do tempo de processamento (2130 ms de um total de 2135 ms, ou seja, mais de 99.7% da execução). Isso ocorre devido à inserção sequencial em múltiplos eixos sem balanceamento prévio por medianas, gerando desequilíbrios severos de profundidade. Em contrapartida, as consultas de busca espacial exata (2 ms) e remoção (3 ms) exibiram altíssima eficiência graças ao particionamento em semi-espacos alternados.

VI. CONCLUSÃO E APLICAÇÕES PRÁTICAS

Como podemos observar ao longo deste estudo, nenhuma estrutura de dados é universalmente superior em todos os

cenários, cargas de trabalho e distribuições de entrada. A escolha ideal depende estritamente do padrão de acesso e do ciclo de vida das operações exigidas pelo problema computacional.

Em termos de aplicações práticas, as características observadas traduzem-se diretamente em domínios reais de computação:

- **Árvores PATRICIA e Trie:** Altamente recomendadas para sistemas que exigem processamento massivo de *strings* e prefixos, sendo aplicadas em tabelas de roteamento IP em roteadores de rede (PATRICIA) e em mecanismos de autocompletar e corretores ortográficos (Trie).
- **Árvores Splay e Treap:** Destacam-se em cenários de busca dinâmica. A Splay é ideal para sistemas com forte localidade temporal de referência, como caches de dados e algoritmos de descompressão adaptativa. A Treap, por sua vez, brilha em filas de prioridade e estruturas aleatorizadas fáceis de manter sem a rigidez de AVL.
- **KD-Tree:** Essencial em problemas de geometria computacional e aprendizado de máquina, sendo aplicada em Sistemas de Informação Geográfica (GIS) para busca de proximidade e em motores de busca de k -vizinhos mais próximos (k -NN).

Portanto, os experimentos práticos consolidados em Algoritmos e Estruturas de Dados demonstram que o projeto de software eficiente exige compreender a fundo os *trade-offs* algorítmicos. Saber ponderar o custo de inserções únicas versus consultas recorrentes é o que dita a escolha técnica correta para cada domínio de aplicação.

REFERÊNCIAS

REFERENCES

- [1] F. Silva, “Implementação de Estruturas de Dados Avançadas: PATRICIA, Trie, Treap e Splay”, Repositório GitHub, 2026. [Online]. Disponível em: <https://github.com/Felipe0132/Trabalho-Arvores-AEDS02>